

Absolutely. Since your project is now working, we can prepare the **Project Documentation** based on the actual system you built rather than a generic template.

AI-Powered Placement Preparation Assistant

Project Documentation

1. Introduction

The **AI-Powered Placement Preparation Assistant** is a web-based application designed to help students prepare for college placement interviews and technical recruitment processes.

The system allows students to:

- Register and securely log in.
- Select or add companies.
- Upload company-specific placement/interview documents in PDF format.
- Extract text from uploaded documents, including scanned PDFs through OCR.
- Convert document content into vector embeddings.
- Store the embeddings in a Qdrant vector database.
- Ask natural-language questions through a chat interface.
- Retrieve relevant information from uploaded documents.
- Generate answers using a Retrieval-Augmented Generation (RAG) architecture.
- Display relevant source pages with the generated answers.

The application combines **React.js**, **Express.js**, **FastAPI**, **PostgreSQL**, **Qdrant**, **Sentence Transformers**, **OCR**, and **Gemini** to provide an interactive placement-preparation platform.

2. Problem Statement

Students preparing for campus placements often need to search through large amounts of information such as:

- Company interview experiences
- Placement brochures
- Job descriptions
- Technical question banks
- DSA preparation material
- Company-specific requirements

Manually searching through these documents can be time-consuming and inefficient.

Traditional chatbots may also provide generic answers that are not directly grounded in the student's uploaded preparation material.

Therefore, the project aims to develop an intelligent assistant that can **retrieve relevant information from placement documents and provide context-based answers with source references**.

3. Objectives

The main objectives of the project are:

1. To develop an AI-powered placement preparation assistant.
 2. To allow students to upload company-specific PDF documents.
 3. To extract text from both normal and scanned PDFs.
 4. To convert document content into vector embeddings.
 5. To store and retrieve document embeddings using Qdrant.
 6. To implement Retrieval-Augmented Generation (RAG).
 7. To provide company-specific document retrieval.
 8. To associate uploaded documents with individual users.
 9. To provide answers with page-level source references.
 10. To provide a user-friendly web interface for placement preparation.
-

4. Scope of the Project

The system focuses primarily on **document-based placement preparation**.

Included

- User authentication
- Company management
- PDF document upload
- OCR-based extraction
- Text chunking
- Embedding generation
- Vector storage
- Semantic search
- RAG-based question answering
- Company-specific retrieval
- User-specific retrieval
- Source/page references
- Chat interface

- PostgreSQL data storage

Possible Future Extensions

- Resume analysis
- Mock interviews
- Automated aptitude tests
- DSA coding practice
- Interview performance tracking
- Personalized preparation plans
- More advanced analytics

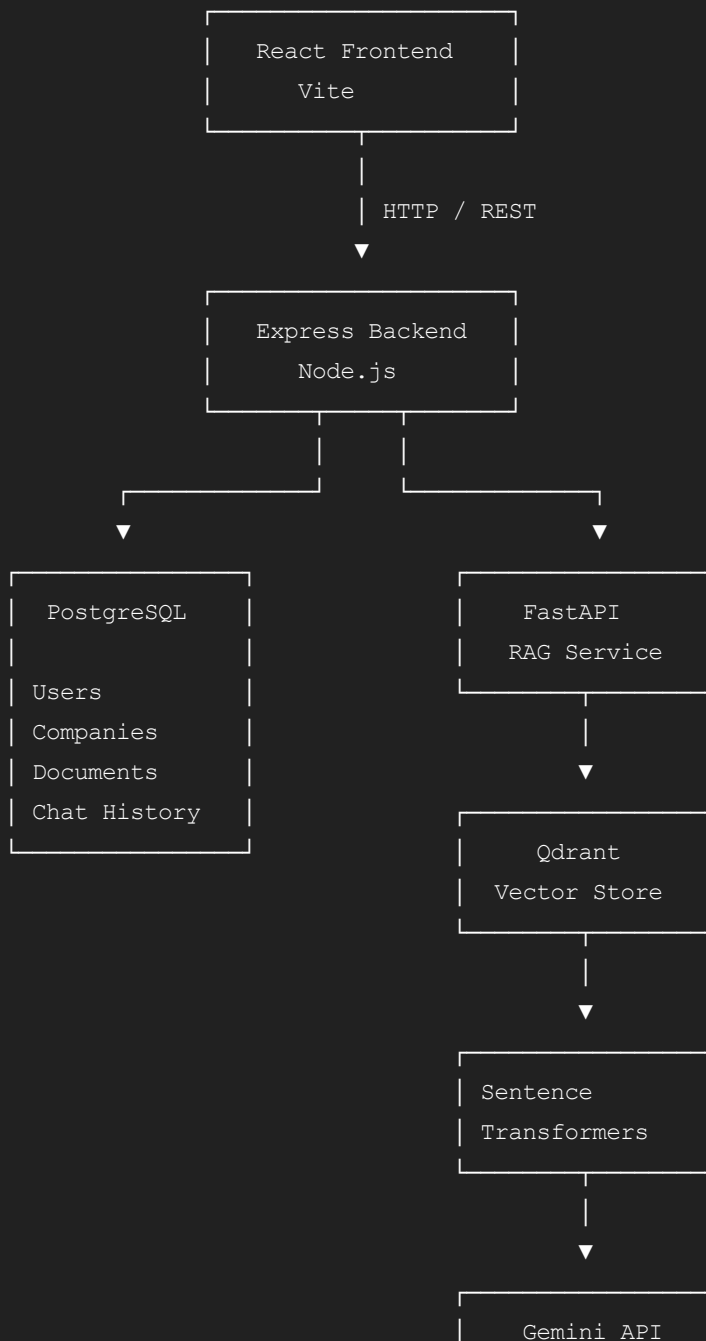
5. Technology Stack

Technology	Purpose
React.js	Frontend user interface
Vite	Frontend development/build tool
Express.js	Backend API server
FastAPI	RAG/AI service
Python	AI and document-processing logic
PostgreSQL	Relational database
Qdrant	Vector database
Sentence Transformers	Text embeddings
<code>all-MiniLM-L6-v2</code>	Embedding model
Gemini API	AI answer generation
LangChain	Text splitting/RAG-related utilities
PyMuPDF	PDF text extraction
Tesseract OCR	Scanned PDF text extraction
Pillow	Image processing for OCR

Technology	Purpose
JWT	Authentication
bcryptjs	Password hashing
Multer	PDF upload handling

6. System Architecture

The application follows a multi-service architecture.



7. Major System Modules

7.1 Authentication Module

The authentication module provides secure registration and login.

Features

- User registration
- User login
- Password hashing using bcrypt
- JWT-based authentication
- Protected API endpoints
- User identification through JWT

The JWT contains the authenticated user's ID, allowing the backend and RAG service to associate requests with the correct user.

8. Company Management Module

The application maintains a `companies` table in PostgreSQL.

Users can:

- View available companies.
- Select a company before uploading a document.
- Add a new company.

Example companies in the system include:

```
TCS
Amazon
Microsoft
Google
Infosys
Accenture
Cloud Analogy
SHNOOR International LLC
```

The company selected during upload is stored with the uploaded document and is also stored as metadata in Qdrant.

9. Document Upload Module

Students can upload PDF preparation material through the React frontend.

The upload process is:

```
Select Company
    ↓
Select PDF
    ↓
React FormData
    ↓
Express /api/documents/upload
    ↓
Multer
    ↓
Store PDF
    ↓
PostgreSQL document record
    ↓
FastAPI RAG service
    ↓
PDF processing
```

The uploaded document is associated with:

- User ID
- Company
- Original file name
- File path
- Upload date

For example:

```
User ID: 9
Company: Test Company
File: Brudite JD.pdf
```

was successfully stored in PostgreSQL during testing.

10. PDF Processing Module

The RAG service processes uploaded PDF documents using PyMuPDF.

For normal PDFs:

```
PDF
↓
PyMuPDF
↓
Extracted Text
```

For scanned/image-based PDFs:

```
PDF
↓
PyMuPDF
↓
No text detected
↓
Render page as image
↓
Tesseract OCR
↓
Extracted Text
```

This allows the system to process both text-based and scanned documents.

11. Text Chunking

Large documents are divided into smaller sections before creating embeddings.

The project uses:

```
Chunk size: 800
Chunk overlap: 150
```

The overlap helps preserve contextual information between neighboring chunks.

Example:

```
Document
↓
Page 1
↓
Chunk 1
Chunk 2
Chunk 3
↓
Page 2
↓
```

Each chunk contains metadata such as:

```
text
page_number
chunk_index
document_name
company
user_id
```

12. Embedding Generation

The project uses the Sentence Transformers model:

```
all-MiniLM-L6-v2
```

Each text chunk is converted into a numerical vector representation.

Conceptually:

```
Text
↓
Sentence Transformer
↓
384-dimensional vector
```

These vectors allow the system to perform **semantic similarity search** rather than relying only on exact keyword matching.

13. Qdrant Vector Database

Qdrant is used as the vector database.

The project uses the collection:

```
placement_documents
```

The collection uses:

```
Vector size: 384
Distance: COSINE
```

Each stored vector contains metadata including:

```
text
page_number
chunk_index
document_name
company
user_id
```

This metadata is important because it allows the system to restrict retrieval according to the authenticated user and selected company.

14. Retrieval Process

When a user asks a question, the question is converted into an embedding.

For example:

```
"What skills are required for this job?"
```

becomes a vector.

Qdrant searches for semantically similar document chunks.

The retrieval process is:

```
User Question
    ↓
Sentence Transformer
    ↓
Question Vector
    ↓
Qdrant Similarity Search
    ↓
Company Filter
    ↓
User Filter
    ↓
Relevant Chunks
```

The system retrieves the most relevant chunks for the question.

15. User and Company-Specific Retrieval

One important feature of the system is that uploaded documents are associated with both:

```
user_id
company
```

For example:

```
User ID = 9
Company = Test Company
```

The retrieval system can therefore retrieve the relevant chunks associated with that user's company-specific documents.

During testing, Qdrant successfully returned results containing:

```
Test Company
User ID 9
```

This confirms that the metadata-based retrieval mechanism is working.

16. Retrieval-Augmented Generation

The project implements a RAG architecture.

RAG combines:

```
Retrieval
+
Generation
```

The complete process is:

```
User Question
    ↓
Question Embedding
    ↓
Qdrant Retrieval
    ↓
Relevant Document Chunks
    ↓
Context Construction
    ↓
Gemini
    ↓
Generated Answer
    ↓
Source References
```

Instead of asking the AI model to answer solely from its general knowledge, the application provides retrieved document content as context.

17. Prompt Design

The RAG prompt instructs the AI assistant to:

- Use the provided context.
- Avoid inventing information.
- State when information cannot be found.
- Provide concise answers.
- Include page references for important claims.

The intended citation format is:

```
[Page X]
```

This helps students trace answers back to the source material.

18. Gemini Integration

Gemini is used as the generative AI component.

The retrieved document chunks are combined into a prompt containing:

```
System instructions
+
Document context
+
User question
```

The Gemini model then generates the final response.

19. Gemini Fallback Mechanism

The project also contains a fallback mechanism for situations where the Gemini API is unavailable or the quota is exhausted.

During testing, Gemini returned:

```
503 UNAVAILABLE
```

and subsequently:

```
429 RESOURCE_EXHAUSTED
```

The application detected the quota condition and returned relevant retrieved document content instead.

Therefore, the application did not completely fail when Gemini was unavailable.

The fallback response indicates:

```
The AI generation service is temporarily unavailable.  
Here is the relevant information retrieved from your documents.
```

This provides graceful degradation.

20. Chat Module

The Chat module provides the primary user interaction with the AI assistant.

Example:

User

```
What skills are required for this job?
```

Assistant

The system retrieved relevant information from the uploaded job description and produced information about:

- Business Development Executive
- Associate Software Engineer
- Content/Social Media Profile

with page references such as:

```
[Page 1]  
[Page 2]  
[Page 3]
```

This demonstrates that the chatbot is using the uploaded document as its information source.

21. Database Design

The project uses PostgreSQL.

Users Table

```
users
-----
id
name
email
password_hash
created_at
role
```

Companies Table

```
companies
-----
id
name
created_at
```

User Documents Table

```
user_documents
-----
id
user_id
company
file_name
file_path
created_at
```

Chat History Table

```
chat_history
-----
id
user_id
question
answer
company
created_at
```

22. API Architecture

Authentication

```
POST /api/auth/register
```

```
POST /api/auth/login
```

User

```
/api/user
```

Chat

```
/api/chat
```

Documents

```
GET /api/documents
```

```
POST /api/documents/upload
```

Companies

```
GET /api/companies
```

```
POST /api/companies
```

RAG Service

```
POST /chat
```

```
POST /documents/process
```

23. Security

The application implements several security mechanisms.

Password Security

Passwords are hashed using:

```
bcryptjs
```

Authentication

JWT tokens are used to authenticate API requests.

Authorization

Protected endpoints require a valid Bearer token.

File Validation

Uploaded files are checked to ensure that:

PDF

files are accepted.

The upload size is limited to:

10 MB

User Isolation

The authenticated user's ID is passed into the RAG retrieval process, helping prevent retrieval of another user's private document chunks.

24. Frontend

The frontend is implemented using React.js and Vite.

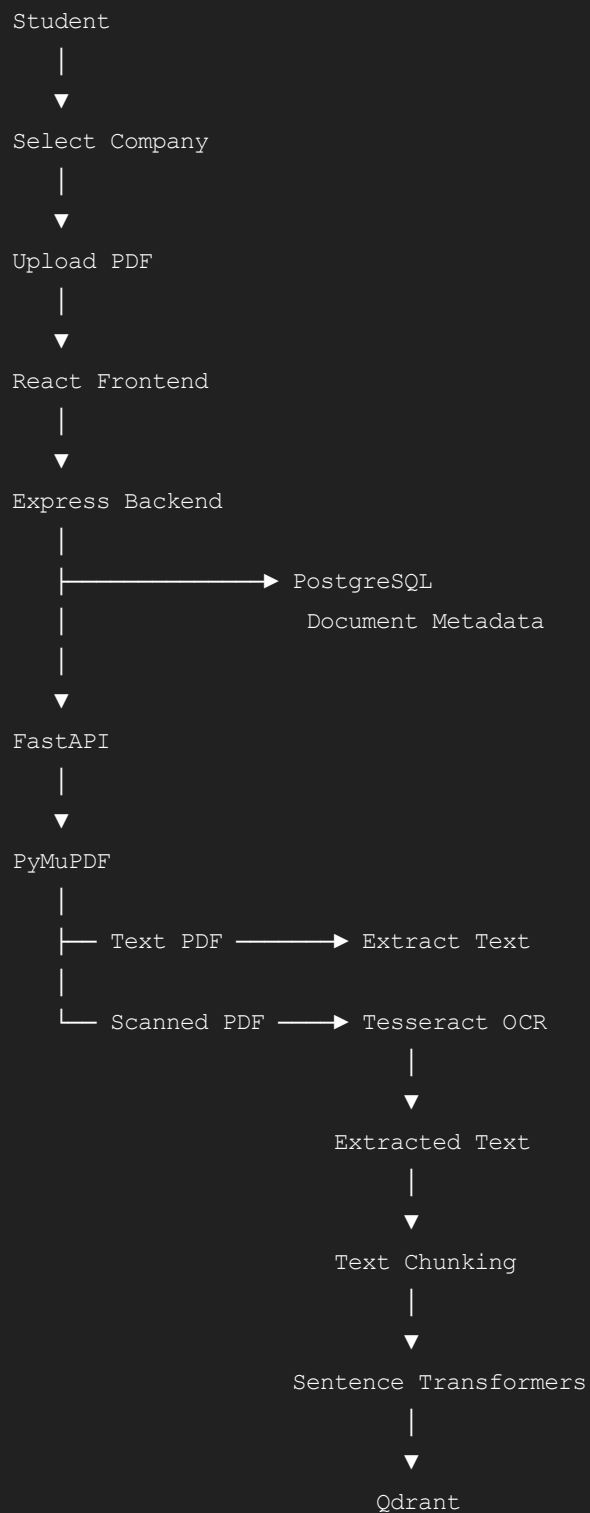
Major UI functionality includes:

- Login
- Registration
- Dashboard
- Company selection
- Add company
- PDF upload
- Drag-and-drop upload
- Upload status
- Chat interface
- AI responses
- Source references

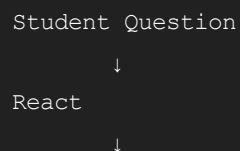
The company selection interface is dynamically loaded from the PostgreSQL-backed Express API rather than relying on hardcoded company names.

25. End-to-End Workflow

The complete document workflow is:



Question workflow:



```
Express
  ↓
FastAPI
  ↓
Create Query Embedding
  ↓
Qdrant Similarity Search
  ↓
Relevant Chunks
  ↓
Gemini
  ↓
Answer
  ↓
Page References
  ↓
React Chat Interface
```

26. Testing

The system was tested component-by-component and end-to-end.

Test	Result
User registration	Pass
User login	Pass
JWT authentication	Pass
Company retrieval	Pass
Add company	Pass
Company selection	Pass
PDF upload	Pass
PostgreSQL document storage	Pass
PDF processing	Pass
OCR processing	Pass
Text chunking	Pass
Embedding generation	Pass

Test	Result
Qdrant storage	Pass
User-specific retrieval	Pass
Company-specific retrieval	Pass
Chat request	Pass
Gemini response	Pass when API available
Gemini quota fallback	Pass
Page citations	Pass
Frontend answer display	Pass

27. Sample Test Result

During testing:

```
User ID: 9
Company: Test Company
Document: Brudite JD.pdf
```

The Qdrant retrieval test returned chunks associated with:

```
Test Company
User ID: 9
```

The Chat interface subsequently returned relevant information from the uploaded job description with page references.

This confirms that the major RAG pipeline is operational.

28. Advantages

The proposed system provides:

1. Faster document-based preparation.
2. Natural-language interaction.

3. Company-specific information retrieval.
 4. User-specific document retrieval.
 5. Support for scanned PDFs through OCR.
 6. Semantic rather than purely keyword-based search.
 7. Source/page references.
 8. Graceful handling of AI API failures.
 9. Centralized company and document management.
 10. An extensible architecture for future placement features.
-

29. Limitations

The current system has some limitations:

1. AI response generation depends on Gemini API availability and quota.
 2. The quality of answers depends on the quality of uploaded documents.
 3. OCR accuracy can vary depending on scanned document quality.
 4. The current system primarily focuses on document-based question answering.
 5. Qdrant is currently running as a local vector database.
 6. The system requires the required backend, RAG service, database, and frontend services to be running.
-

30. Future Scope

Future versions can include:

Resume Analysis

Students could upload resumes and receive:

- Skill extraction
- Skill-gap analysis
- Job-description matching
- Resume improvement suggestions

Mock Interviews

The system could conduct interactive mock interviews and evaluate answers.

DSA Practice

The assistant could generate:

- Coding questions
- Hints

- Explanations
- Difficulty levels
- Company-specific DSA practice

Aptitude Preparation

The system could provide:

- Quantitative aptitude
- Logical reasoning
- Verbal ability
- Timed tests

Personalized Preparation

The system could create a personalized preparation plan based on:

```
Student skills
+
Target company
+
Job description
+
Preparation history
```

31. Conclusion

The **AI-Powered Placement Preparation Assistant** provides an intelligent document-based platform for placement preparation.

The system integrates a React frontend, Express backend, FastAPI RAG service, PostgreSQL database, Qdrant vector database, Sentence Transformers, OCR, and Gemini.

The implemented RAG pipeline allows students to upload placement-related documents and ask natural-language questions about them. Relevant document chunks are retrieved using semantic similarity and filtered according to company and user metadata. The retrieved information is then used to generate answers with page references.

Testing has demonstrated that the complete workflow—from **PDF upload to document processing, vector storage, retrieval, and Chat-based response generation—is functioning successfully.**

The project therefore provides a foundation that can be extended with resume analysis, mock interviews, DSA practice, aptitude testing, and personalized placement preparation.